INKFORGE: Build Documentation

A fully local writing system combining visual planning, staged generation, and autonomous quality control

Purpose. INKFORGE is a proposed architecture that combines the strongest ideas represented in the three supplied diagrams: OpenWrite-style visual planning and a searchable Codex; GOAT-style top-down staged planning and resumable scene generation; and AuthorAgent-style orchestration, continuity checks, character-voice critics, revision loops, skills, and export tooling.

This is a new synthesis, not a claim that the three upstream projects already implement every feature. Treat the SVG as the target architecture and this document as the implementation plan.

1. Product goals and non-negotiable constraints

- **Local-only runtime:** application, database, indexes, model inference, logs, and exports stay on the user's machine.
- **Offline after setup:** download application dependencies and model weights in a deliberate setup phase; runtime must work with the network disconnected.
- **Author remains in control:** AI output is a proposal. Canon changes, destructive operations, and publication exports require review or explicit confirmation.
- **Long-book reliability:** every pipeline stage writes a checkpoint

and can be run independently, edited manually, retried, or resumed.
- **Inspectable reasoning artifacts:** outlines, scene cards, continuity findings, prompts/config, and revision diffs are persisted.
- **Portable project data:** plain Markdown/JSON files are the source of truth; database and search indexes are rebuildable.

2. Recommended stack

Layer	Recommendation	Why
UI	React + TypeScript + Vite	Fast local web UI; good ecosystem for graph and editor components
Story map	React Flow (`@xyflow/react`)	Node-based plot, character, location, and dependency maps
Manuscript editor	Tiptap or CodeMirror	Rich text or Markdown editing with local autosave
API	Python + FastAPI	Strong fit for orchestration, validation, NLP and model tooling
CLI	Typer	One command interface for stages, checks, exports and batch runs
Workflow	Python state machine / explicit stage runner	Clear checkpoints, retries, pause/resume and stage isolation
Metadata DB	SQLite + SQLModel or SQLAlchemy	Single-file local persistence and migrations

Lexical search	SQLite FTS5	Simple, local, robust exact/term search
Optional vector retrieval	`sentence-transformers` + FAISS or Chroma local mode	Semantic recall when hardware permits; rebuildable index
Model runtime	Ollama first; llama.cpp server adapter second	Simple local inference; GGUF model support and CPU/GPU backends
Exports	python-docx, Pandoc (optional), ebooklib	DOCX/EPUB/Markdown outputs; validate each export format
Packaging	`uv` + `pyproject.toml`; Docker optional	Reproducible Python environment; Docker can be used without networking at runtime

Default deployment: FastAPI and the model runtime listen only on loopback (`127.0.0.1`). The React app is served locally. Docker is optional; do not require a container if it complicates GPU or Apple Metal access.

3. System boundaries and data flow

1. The author creates a project in the UI or CLI and fills a **Book Spec**.
2. The Conductor runs one stage at a time: spec refinement → story design → chapter outline → scene cards → scene drafting → critique → revision → export.
3. The Context Builder assembles a small, relevant prompt from the Book Spec, Codex, CORE memory, current chapter/scene plan, selected prior scenes and retrieval results.
4. The local model router sends inference requests to the selected local runtime. There is no remote-provider fallback.
5. The output is schema-validated and saved as a new version. Invalid output is retried or surfaced for correction; it does not

replace the last valid artifact.
6. Deterministic validators run first. Local-model critics can classify ambiguous cases, but findings must point to evidence and remain advisory.
7. The author can edit any artifact and resume from that checkpoint.
8. Export builds the requested manuscript from approved scene versions.

Trust model

- **Deterministic application code:** authoritative for file paths, schemas, project IDs, versioning, checkpoints, audit records, and export assembly.
- **LLM:** untrusted generator and reviewer. It may hallucinate facts, so never treat model claims as canon without validation.
- **Canon:** changes only through an explicit author action or a configured, reviewable operation.
- **Indexes/summaries:** disposable derived artifacts. The original manuscript and structured records are the source of truth.

4. Suggested repository layout

```
inkforge/
├── apps/
│   ├── api/                    # FastAPI app and depende
│   └── web/                    # React + Vite UI
├── packages/
│   ├── domain/                 # Pydantic models and dom
│   ├── pipeline/               # stage runner, checkpoin
│   ├── context/                # context assembly and to
│   ├── memory/                 # summaries, FTS5, option
│   ├── quality/                # continuity, voice, craf
│   ├── models/                 # Ollama / llama.cpp adap
│   ├── prompts/                # versioned prompt and sk
│   └── exporters/              # Markdown, DOCX, EPUB an
├── cli/                        # Typer commands
```

```
├── tests/
│   ├── unit/
│   ├── integration/
│   └── fixtures/
├── config/
│   ├── defaults.toml
│   └── model-profiles.toml
├── workspace/                    # user data; keep out of
├── scripts/
├── pyproject.toml
└── README.md
```

Keep user manuscripts and model weights outside the application repository. Add `workspace/`, model caches, secrets, logs, generated books, and local config overrides to `.gitignore`.

5. Core data model

Use Pydantic models as the shared contract between pipeline stages and UI. Persist structured records as versioned JSON as well as indexing project metadata in SQLite.

Minimum entities:

- **Project:** id, title, genre, status, created/updated timestamps, active version.
- **BookSpec:** premise, audience, genre, tone, POV, tense, content constraints, target length, narrative promises.
- **Character:** id, canonical name, aliases, role, goals, fears, secrets, traits, voice card, knowledge state, relationships.
- **Location** / **LoreFact:** id, description, rules, first introduced, source references, confidence, canon status.
- **PlotBeat:** id, purpose, act, setup/payoff links, dependencies, status.
- **ChapterPlan:** id, goal, conflict, turning point, outcome, pacing note, required continuity.
- **SceneCard:** id, chapter id, POV, location, participants, want, obstacle, change, mood, beats, exit condition.

- **SceneVersion:** scene id, version, text path, model profile, prompt version, input hashes, created time.
- **Finding:** category, severity, claim, evidence spans, affected entities, confidence, status, resolution.
- **Checkpoint** / **Job:** stage, input artifact hashes, output paths, status, attempts, timestamps, error summary.
- **AuditEvent:** operation, target, actor (`user` or `system`), timestamp, input/output references.

Never rely on a vector index as the only place a fact exists. Persist canon facts in structured records and cite the original scene or author action that established them.

6. Pipeline implementation

Implement each stage as a function with typed inputs and outputs. Keep orchestration separate from generation logic.

```
class Stage:
    name: str

    def validate_input(self, context): ...
    def build_prompt(self, context): ...
    def generate(self, prompt, model): ...
    def validate_output(self, result): ...
    def persist_version(self, result, metadata): ...
```

The above is an interface sketch, not complete production code. Each concrete stage should be deterministic except for its model call.

Stage A — Book Spec

Capture premise, genre, tone, audience, POV, tense, target length, themes, content boundaries, and non-negotiable constraints. Offer an optional spec-enhancement pass, but preserve the user's original fields and show changes as a diff.

Stage B — Story Design

Generate the central dramatic question, theme, promise to the reader, major arcs, acts, turning points, setups/payoffs, and ending constraints. Every important payoff should link to a setup or be flagged as intentionally surprising/unseeded.

Stage C — Chapter Outline

Each chapter should include purpose, conflict, change, causal link to prior/next chapters, characters present, required information, and intended pacing. Check for duplicate chapter functions and missing causal transitions.

Stage D — Scene Cards

Split chapters into scenes with POV, location, participants, character wants, opposition, emotional shift, concrete event, scene turn, and exit condition. Scene cards are editable before prose generation.

Stage E — Scene Drafting

Draft one scene at a time. Include the scene card, relevant Codex entries, a compact chapter/arc digest, and only the prior-scene material needed for continuity. Detect truncation and continue safely without duplicating the last paragraph. Save draft versions rather than overwriting.

Stage F — Critique

Run deterministic checks before model critique:

- required scene elements and schema validity;
- known facts (names, ages, dates, locations, object states);
- timeline and character knowledge constraints;

- plot promise/setup/payoff links;
- repeated phrases and simple style linting.

Then use specialized local critics for voice, motivation, pacing, clarity, prose quality, and reader reaction. Each finding must include evidence or be explicitly marked as a hypothesis.

Stage G — Revision

Turn accepted findings into small, scoped edit requests. Produce a diff, not an opaque rewrite. Re-run the relevant checks after each revision. Limit loops by configurable maximum attempts and stop when the rubric plateaus or the author rejects the change.

Stage H — Export

Assemble approved scene versions in outline order. Support Markdown first, then DOCX and EPUB. Add a preflight report for missing scenes, unresolved high-severity continuity findings, empty chapters, duplicate IDs, and unsupported export features. Do not imply that generated output is publisher-ready without human proofreading.

7. Memory and context strategy for long books

Use three explicit tiers:

1. **CORE:** short, stable facts always included (POV/tense, key character traits, current timeline anchor, style constraints, canon rules).
2. **DIGEST:** chapter and arc summaries, updated after a scene or chapter is approved.
3. **RECALL:** exact source passages retrieved on demand with SQLite FTS5; optionally combine with local embeddings and a vector index.

A useful context assembly order is:

- system constraints and output schema;
- Book Spec and current stage instructions;
- CORE facts;
- current chapter outline and scene card;
- relevant Codex entries;
- top matching source passages;
- compact recent-scene recap;
- output contract.

Reserve a configurable fraction of the model context for the generated output. Count tokens with the selected model's tokenizer when available. If the prompt is too large, remove low-relevance recall passages before truncating canon or the current scene card. Store the assembled context manifest with every run so a result can be debugged or reproduced.

8. Continuity engine design

Use a hybrid approach. Start with explicit facts and rules rather than asking an LLM to reread an entire novel.

- Extract candidate facts from approved scenes into a **review queue**.
- Store accepted facts with entity ID, predicate, value, time scope, source passage, and canon status.
- Run deterministic checks for incompatible values, chronology, scene presence, travel/time plausibility (if configured), character knowledge, and object ownership/state.
- Use a local critic only when rules cannot determine whether two facts conflict.
- Group findings by severity and affected scene; show source evidence and links to both passages.
- Allow author resolutions: confirm contradiction, mark intentional, update canon, or dismiss.
- Do not auto-edit manuscripts based solely on a contradiction score.

9. Visual planning UI

The graph is not just decoration; it should be backed by typed entities and links.

Recommended node types: premise, act, plot beat, chapter, scene, character, location, lore fact, setup, payoff. Recommended edges: contains, precedes, depends-on, introduces, pays-off, appears-in, knows, located-at.

Views:

- **Story Map:** graph with filters and zoomable clusters.
- **Outline:** sortable hierarchical list, drag/reorder with validation.
- **Timeline:** dated events and character presence.
- **Codex:** character/lore/location pages and cross-links.
- **Manuscript:** editor with scene status and continuity markers.
- **Run Console:** stage events, errors, checkpoints, model profile, and context size.
- **Quality Desk:** findings with evidence, severity, status, and resolution controls.

Graph changes should update the structured database and be journaled. Keep a keyboard-accessible outline view for users who do not want to use a canvas.

10. Local model routing

Use a single internal interface such as `generate(messages, model_profile, max_tokens, temperature)`. Implement adapters for Ollama and llama.cpp OpenAI-compatible endpoints.

Model profiles can differ by task:

- **planner:** follows structured JSON and handles long instructions;
- **prose:** produces fluent scene drafts;
- **critic:** evaluates against a rubric and returns evidence-linked

findings;
- **embedding:** optional local embedding model.

Do not require four different models; one capable local model can serve every role on constrained hardware. Model routing is a configuration choice. The router should expose health, context size, queue state, timeout, cancellation and error details.

Hardware reality: model memory needs vary significantly by architecture, quantization, context length, and runtime. A quantized 7B–8B model is a practical starting point on many machines, but long-range literary consistency will be weaker than with a stronger model. Larger models require substantial RAM/VRAM. Benchmark candidate models on a fixed test book before committing.

11. Local-only security checklist

- Bind API and model server to `127.0.0.1`, not `0.0.0.0`, unless the user deliberately enables LAN access.
- Disable telemetry, remote API providers, online search tools, automatic update checks, and cloud fallbacks.
- Make outbound network access an OS firewall rule or container policy if strict offline guarantees are required. “Local-first” code alone is not a network isolation guarantee.
- Download models and dependencies during a clearly separated setup step; record versions and checksums where possible.
- Validate all paths against the selected project root; reject traversal and symlink escapes.
- Keep secrets out of project files and logs. The default design should not need API keys.
- Write append-only audit events for file modifications, revisions, imports, and exports.
- Use atomic writes and backups before bulk operations.
- Do not execute generated code, shell commands, macros, or arbitrary plugin scripts from manuscript content.
- Test in airplane mode and inspect network sockets before claiming offline readiness.

12. Build roadmap

Milestone 1 — Local skeleton

Create the repo, FastAPI service, React UI, SQLite database, project/workspace manager, local model adapter, and CLI. Verify it starts without internet once dependencies are installed.

Milestone 2 — Structured planning

Implement Pydantic schemas, Book Spec, story design, chapter outline, scene cards, manual editing, JSON/Markdown persistence, and per-stage checkpoints. Make every stage runnable independently.

Milestone 3 — Drafting workflow

Add scene drafting, context budgeting, truncation detection, resumable jobs, version history, and diff review.

Milestone 4 — Visual workbench and Codex

Build graph and outline views, character/location/lore records, links between beats and chapters, and autosave.

Milestone 5 — Quality system

Implement deterministic continuity checks, evidence-linked findings, character voice profiles, critique rubric, revision proposals, and review controls.

Milestone 6 — Long-book memory

Add CORE/DIGEST/RECALL, FTS5, source references, optional local

embeddings, and context manifests.

Milestone 7 — Export and hardening

Add Markdown/DOCX/EPUB exports, preflight checks, backups, migration tests, local-only security tests, and offline acceptance tests.

Milestone 8 — Packaging

Provide a one-command launcher, setup wizard for local model selection, hardware guidance, sample project, troubleshooting, and backup/restore instructions.

13. Testing and acceptance criteria

A first release should not be called complete until these pass:

- **Offline test:** after setup, airplane-mode startup, project creation, planning, drafting, critique, and export all work.
- **Resume test:** terminate a long run mid-pipeline; restart and resume from the last valid checkpoint without corrupting artifacts.
- **Stage isolation:** run every stage individually against saved inputs.
- **Canon test:** contradictory facts produce evidence-linked findings; no canon change occurs without confirmation.
- **Path safety test:** traversal and symlink escape attempts are rejected.
- **Context test:** overlong context is trimmed according to priority and the context manifest records what was included.
- **Export test:** exported chapter order matches the approved outline and no unapproved draft version is included.
- **Data portability test:** a project can be backed up, restored, and opened on another machine with the same app version.
- **Network test:** application configuration contains no cloud fallback and OS/container policy blocks outbound requests during the offline test.

14. Suggested initial commands

These are example commands to aim for; implement them as the CLI evolves.

```
# install development dependencies during setup
uv sync
cd apps/web && npm install && npm run build

# start the local API and UI
uv run uvicorn apps.api.main:app --host 127.0.0.1 --port
cd apps/web && npm run dev -- --host 127.0.0.1

# planned CLI interface
uv run inkforge init "The Glass Orchard"
uv run inkforge plan --project the-glass-orchard
uv run inkforge run-stage draft-scene --project the-glass
uv run inkforge check --project the-glass-orchard
uv run inkforge export --project the-glass-orchard --form
```

The commands above are target UX examples, not runnable commands until the corresponding modules and CLI entry points are implemented.

15. What to build first

Build the **data model + checkpointed pipeline before the graph UI**. A beautiful story map cannot compensate for weak state management. The first end-to-end vertical slice should create a Book Spec, produce a chapter outline, generate one scene, run one continuity check, persist a checkpoint, and export Markdown entirely offline. Then layer in the visual graph, Codex, richer critics, reader panels, and additional export formats.

16. Relationship to the supplied designs

- **OpenWrite diagram:** informs the local web workspace, story-map graph, Codex, editor, streaming assistant, and local database/runtime pattern.
- **GOAT-Storytelling-Agent diagram:** informs the strict top-down sequence, scene cards, independent stage execution, local GGUF inference, context trimming, and checkpoint/resume behavior.
- **AuthorAgent diagram:** informs the Conductor, skills registry, long-book memory tiers, continuity engine, character-specific voice critics, revision loop, reader panels, audit trail, and export/publishing suite.

The architecture deliberately keeps these capabilities behind shared schemas and services so they can work together rather than becoming three separate applications.